

Portfolio Optimization Suite

Modern Portfolio Theory, Black-Litterman & Risk-Parity with
Interactive Visualisations

Tech Stack: Python · NumPy · CVXPY · Dash

Version v1.0 · June 2026 · Internal Engineering Specification

Table of Contents

#	Section	Description
1	Project Overview	Goals, scope, supported models
2	System Architecture	Module structure, data flow
3	Data Layer	Returns data, factor loading, covariance estimation
4	Optimisation Engine — MPT	Mean-variance, efficient frontier
5	Optimisation Engine — Black-Litterman	Views, posterior, BL weights
6	Optimisation Engine — Risk Parity	Equal risk contribution, HRP
7	Constraints & Regularisation	Turnover, leverage, sector limits
8	Interactive Dashboard — Dash	Layout, callbacks, visualisations
9	Performance Attribution	Brinson attribution, factor decomposition
10	Testing & Numerical Validation	Convexity checks, solver correctness

1. Project Overview

The **Portfolio Optimization Suite** is a Python-based quantitative research tool that implements three industry-standard portfolio construction methodologies — Mean-Variance (MPT), Black-Litterman, and Risk Parity — with a Plotly Dash frontend for interactive exploration of the efficient frontier and weight allocation.

Language	Python 3.11+
Core Libraries	NumPy 1.26, SciPy 1.13, CVXPY 1.4, pandas 2.2, Plotly/Dash 2.17
Solvers	CLARABEL (default), MOSEK (optional, faster for large universes)
Universe	Up to 500 assets (tested; theoretical limit solver-dependent)
Data Sources	yFinance, CSV upload, Bloomberg (optional via blpapi)

1.1 Supported Optimisation Models

Model	Objective	Key Inputs
Mean-Variance (Markowitz)	Maximise Sharpe / Minimise Variance	Expected returns, covariance matrix
Black-Litterman	Posterior expected returns from investor views	Market caps, views, view confidence
Risk Parity (ERC)	Equal risk contribution across assets	Covariance matrix only
Hierarchical Risk Parity	Cluster-based diversification	Correlation matrix, linkage method
Maximum Diversification	Maximise diversification ratio	Volatilities, correlation matrix

2. System Architecture

2.1 Module Structure

```
// Project structure
portfolio_suite/
  data/
    loaders.py # yFinance, CSV, Bloomberg adapters
    returns.py # Log/arithmetic return calculation
    covariance.py # Sample, Ledoit-Wolf, factor-model shrinkage
  optimisation/
    base.py # BaseOptimiser ABC
    mean_variance.py # Markowitz MPT + efficient frontier
    black_litterman.py # BL posterior estimation
    risk_parity.py # ERC + HRP
    constraints.py # Turnover, sector, leverage constraints
  attribution/
    brinson.py # Allocation/selection/interaction
    factor.py # Fama-French factor decomposition
  dashboard/
    app.py # Dash application entry point
    layouts/ # Page layouts
    callbacks/ # Interactivity callbacks
  tests/
    test_optimisers.py
    test_numerical.py
```

3. Data Layer

3.1 Returns Calculation

```
// Return calculation
# data/returns.py
def compute_returns(prices: pd.DataFrame,
                    method: Literal['log', 'arithmetic'] = 'log',
                    frequency: Literal['daily', 'weekly', 'monthly'] = 'daily'
) -> pd.DataFrame:
    if frequency == 'weekly':
        prices = prices.resample('W-FRI').last()
    elif frequency == 'monthly':
        prices = prices.resample('ME').last()
    if method == 'log':
        return np.log(prices / prices.shift(1)).dropna()
    return prices.pct_change().dropna()
```

3.2 Covariance Estimation Methods

Method	When to Use	Implementation
Sample Covariance	$N \gg p$, long history available	<code>np.cov(returns.T)</code>
Ledoit-Wolf Shrinkage	Default; N moderately larger than p	<code>sklearn.covariance.LedoitWolf</code>
Constant Correlation	When individual correlations unreliable	Custom implementation
Factor Model	Large universe (> 100 assets)	Fama-French 5-factor + residual

4. Optimisation Engine — MPT

4.1 CVXPY Problem Formulation

```
// CVXPY minimum variance
# optimisation/mean_variance.py
def min_variance_portfolio(mu: np.ndarray, Sigma: np.ndarray,
constraints: list) -> OptResult:
    n = len(mu)
    w = cp.Variable(n)
    portfolio_variance = cp.quad_form(w, Sigma)
    base_constraints = [
        cp.sum(w) == 1,
        w >= 0, # long-only; remove for long-short
    ]
    prob = cp.Problem(cp.Minimize(portfolio_variance),
        base_constraints + constraints)
    prob.solve(solver=cp.CLARABEL)
    if prob.status not in ['optimal', 'optimal_inaccurate']:
        raise OptimisationError(f'Solver failed: {prob.status}')
    return OptResult(weights=w.value, portfolio_variance=prob.value,
        expected_return=float(mu @ w.value))
```

4.2 Efficient Frontier Generation

- Parametric sweep: 100 points across target return range [min-variance return, max-return asset return]
- Each point solves a minimum-variance problem with equality constraint on expected return
- Output: array of (risk, return, weights) tuples stored for dashboard rendering
- Critical portfolios computed: min-variance, max-Sharpe, max-return

5. Optimisation Engine — Black-Litterman

5.1 BL Model Implementation

The Black-Litterman model blends market-implied returns (reverse-optimised from cap weights) with investor views:

```
// Black-Litterman posterior
# optimisation/black_litterman.py
def black_litterman_posterior(
    Sigma: np.ndarray, # NxN covariance
    cap_weights: np.ndarray, # market cap weights
    P: np.ndarray, # KxN pick matrix (K views)
    Q: np.ndarray, # K view returns
    Omega: np.ndarray, # KxK view uncertainty (diagonal)
    delta: float = 2.5, # risk-aversion coefficient
    tau: float = 0.05, # scaling factor
) -> tuple[np.ndarray, np.ndarray]:
    # Step 1: Implied equilibrium returns
    pi = delta * Sigma @ cap_weights
    # Step 2: Posterior expected returns
    M_inv = np.linalg.inv(tau * Sigma)
    BL_mu = np.linalg.solve(
        M_inv + P.T @ np.linalg.solve(Omega, P),
        M_inv @ pi + P.T @ np.linalg.solve(Omega, Q)
    )
    BL_Sigma = np.linalg.inv(M_inv + P.T @ np.linalg.solve(Omega, P))
    return BL_mu, BL_Sigma
```

5.2 View Input Interface

- **Absolute views:** 'AAPL will return 12% p.a.' — diagonal P, scalar Q
- **Relative views:** 'AAPL will outperform MSFT by 2%' — P row: [+1, -1, 0...]
- **Confidence:** Idzorek method — specify % confidence, Omega computed automatically

6. Optimisation Engine — Risk Parity

6.1 Equal Risk Contribution (ERC)

ERC finds weights such that each asset contributes equally to total portfolio risk:

```
// ERC optimisation via SciPy
# optimisation/risk_parity.py
def erc_objective(w: np.ndarray, Sigma: np.ndarray) -> float:
    port_var = w @ Sigma @ w
    mrc = Sigma @ w # marginal risk contributions
    rc = w * mrc # risk contributions
    return float(np.sum((rc - port_var / len(w))**2))

def equal_risk_contribution(Sigma: np.ndarray) -> np.ndarray:
    n = len(Sigma)
    w0 = np.ones(n) / n # equal weight starting point
    bounds = [(0.001, 1.0)] * n
    constraints = [{'type': 'eq', 'fun': lambda w: np.sum(w) - 1}]
    res = minimize(erc_objective, w0, args=(Sigma,),
        method='SLSQP', bounds=bounds,
        constraints=constraints, tol=1e-10)
    return res.x / res.x.sum()
```

6.2 Hierarchical Risk Parity (HRP)

- **Step 1: Tree clustering** — hierarchical clustering of assets by correlation distance ($1 - |\rho|$)
- **Step 2: Quasi-diagonalisation** — reorder covariance matrix per cluster tree
- **Step 3: Recursive bisection** — allocate weight top-down, splitting by inverse-variance within each cluster
- Implemented using `scipy.cluster.hierarchy` and custom recursive bisection

7. Constraints & Regularisation

7.1 Supported Constraints (CVXPy)

```
// Constraint builder
def build_constraints(w, w_prev, config: ConstraintConfig) -> list:
    c = [cp.sum(w) == 1]
    if config.long_only: c.append(w >= 0)
    if config.max_weight: c.append(w <= config.max_weight)
    if config.min_weight: c.append(w >= config.min_weight)
    if config.max_turnover and w_prev is not None:
        c.append(cp.norm1(w - w_prev) <= config.max_turnover)
    if config.sector_limits:
        for sector, (lo, hi) in config.sector_limits.items():
            mask = np.array([1 if a in sector_map[sector] else 0
                             for a in assets])
            c.append(mask @ w >= lo)
            c.append(mask @ w <= hi)
    return c
```


8. Interactive Dashboard — Dash

8.1 Dashboard Panels

- **Asset Universe Builder:** ticker search, upload, date range picker
- **Efficient Frontier:** interactive scatter (risk vs return); hover shows full weights
- **Weight Allocation:** bar chart + treemap of portfolio weights by sector
- **Correlation Heatmap:** asset return correlations, sortable by cluster
- **Risk Contribution Chart:** stacked bar of risk contribution per asset
- **BL View Editor:** form to add/remove/weight investor views, live posterior update
- **Backtested Performance:** cumulative return of optimised portfolio vs benchmark
- **Attribution Summary:** Brinson allocation/selection table for period

9. Performance Attribution

9.1 Brinson Attribution

```
// Brinson attribution model
# attribution/brinson.py
def brinson_attribution(portfolio_weights, benchmark_weights,
portfolio_returns, benchmark_returns) -> Attribution:
allocation = (portfolio_weights - benchmark_weights) * benchmark_returns
selection = benchmark_weights * (portfolio_returns - benchmark_returns)
interaction = (portfolio_weights - benchmark_weights) * (portfolio_returns -
benchmark_returns)
total_active = allocation + selection + interaction
return Attribution(allocation=allocation, selection=selection,
interaction=interaction, total=total_active)
```

10. Testing & Numerical Validation

10.1 Convexity & Correctness Checks

- **Efficient frontier monotonicity:** assert risk increases monotonically with target return across all 100 frontier points
- **Weight sum:** assert $|\text{sum}(w) - 1| < 1e-6$ for every optimisation result
- **Non-negativity:** assert $\text{all}(w \geq -1e-8)$ for long-only portfolios
- **ERC equality check:** assert $\text{std}(\text{risk_contributions}) < 1e-4$ for ERC solution
- **BL boundary cases:** zero-view portfolio must equal reverse-optimised equilibrium weights
- **Covariance PSD check:** assert all eigenvalues ≥ 0 after shrinkage

10.2 Reference Portfolio Tests

- 60/40 portfolio (SPY/TLT) reproduced to 4 decimal places vs published weights
- Fama-French tangency portfolio on 10-industry dataset matches published results within 0.5%
- HRP on Lopez de Prado's example dataset matches reference implementation exactly